

Модуль 7. Redis Streams: команды XADD, XREAD и архитектура Append-Only Log

7.1. Зачем Redis Streams, если есть обычные списки?

В прошлом модуле мы построили модель ресторана на Python `Queue`. В Модуле 5 мы уже видели Redis, но использовали его как «чёрный ящик» для счётчиков (`XLEN`) и простых записей. Теперь пришло время понять, **как именно** Redis хранит сообщения, и почему для серьёзных систем используется не просто «список», а специальная структура — **Streams**.

7.1.1. Классические списки Redis: мгновенный обзор

До появления Streams (Redis 5.0, 2018 год) в Redis для очередей использовали **списки** (Lists):

- `LPUSH myqueue "message"` — положить сообщение в начало списка.
- `BRPOP myqueue 0` — заблокироваться и ждать сообщение из конца списка.

Проблема: Как только вы выполняете `BRPOP`, сообщение **исчезает** из списка. Если Consumer взял сообщение, начал обрабатывать, и в этот момент упал — сообщение потеряно. Нет механизма «вернуть обратно». Нет истории. Нет параллельного чтения несколькими группами.

Аналогия: Лист бумаги с заказами, приклеенный скотчем к стене. Официант отрывает верхний листок, несёт на кухню. Если листок потерялся по дороге — заказ забыт. Другие повара не знают, что было написано.

7.1.2. Что нужно настоящей системе

Настоящему брокеру сообщений нужно:

1. **Сообщения не исчезают** после прочтения — они остаются в журнале.
2. **Несколько независимых читателей** могут читать один и тот же поток. Например, один сервис строит отчёты, другой — шлёт уведомления, третий — пишет в архив.
3. **Группы потребителей** могут делить сообщения между собой: три воркера обрабатывают поток параллельно, но одно сообщение не попадает двум воркерам одновременно.
4. **Подтверждение обработки (ACK)** — если воркер упал, сообщение возвращается в очередь группы.

Redis Streams решает все эти задачи.

7.2. Что такое Append-Only Log: фундаментальная концепция

7.2.1. Определение

Append-Only Log (журнал с добавлением только в конец) — это структура данных, в которую новые записи **всегда** добавляются строго в конец. Старые записи **не изменяются** и **не удаляются** (по крайней мере, не в рамках обычной работы).

Аналогия: бухгалтерская книга или судовый журнал

Представьте капитана корабля XVIII века. Каждый вечер он открывает судовый журнал и записывает: «20 августа. Широта 45°. Ветер северный. Увидели китов». Он **никогда не вырывает** старые страницы. Он **никогда не исправляет** запись за вчерашний день карандашом. Он только **дописывает** новые строки в конец.

Если капитан ошибся вчера — он пишет новую строку: «21 августа. Уточнение: вчерашняя широта была 46°, а не 45°». Старая запись остаётся. Вся история сохранена.

Свойства Append-Only Log:

- **Персистентность:** записи хранятся на диске (в Redis — через механизмы RDB/AOF).
- **Неизменяемость:** запись, попавшая в лог, не редактируется.
- **Упорядоченность:** каждая запись имеет строгий порядковый номер (в Redis — это ID на основе времени).
- **Множественное чтение:** десять человек могут читать один и тот же журнал, не мешая друг другу.

7.3. Устройство Redis Streams

7.3.1. Структура записи в Stream

Каждая запись (entry) в Stream — это:

```
In [1]: ID: 1724169600000-0
      │ field1: value1
      │ field2: value2
      └─ field3: value3
```

ID — это не просто порядковый номер. Это **составной идентификатор**:

```
In [2]: 毫秒时间戳-序列号
```

Например: 1724169600000-0

- `1724169600000` — timestamp в миллисекундах (Unix-время).
- `0` — порядковый номер сообщения внутри этой миллисекунды (если за одну миллисекунду пришло несколько сообщений: `0` , `1` , `2` ...).

Зачем такая сложность?

- ID автоматически сортирует сообщения по времени.
- Можно искать сообщения за конкретный временной диапазон.
- Гарантируется уникальность без центрального координатора.

7.3.2. Ключевое отличие от очереди

Классическая очередь (List)	Redis Stream
Сообщение удаляется при чтении	Сообщение остаётся в логе
Один прочитал — другой не видит	Любое количество читателей
Нет истории	Есть полная история
Нет ACK	Есть подтверждение обработки (в группах)
Простая	Богатая семантика

7.4. Команды Redis Streams: полный разбор

Мы разберём команды в логическом порядке: сначала пишем, потом читаем, потом управляем группами.

7.4.1. `XADD` — добавить сообщение в Stream

Синтаксис:

```
In [3]: XADD имя_потока [MAXLEN ~ лимит] [NOMKSTREAM] ID поле значение [поле значение ...]
```

Практика в `redis-cli` :

Подключитесь к Redis из контейнера (если у вас остался проект из Модуля 5):

```
In [4]: docker compose exec redis redis-cli
```

Или запустите Redis локально:

```
In [5]: docker run -d --name redis_for_learning -p 6379:6379 redis:7-alpine
        docker exec -it redis_for_learning redis-cli
```

Теперь добавим сообщения:

```
In [6]: XADD events:users * action login user_id 42 ip 192.168.1.1
```

Разбор:

- `events:users` — имя потока (как имя таблицы в БД).
- `*` — сказать Redis: «сгенерируй ID сам». Redis вернёт что-то вроде `1724169600000-0`.
- `action login user_id 42 ip 192.168.1.1` — пары поле-значение. Это не JSON, это плоская структура ключ-значение.

Результат:

```
In [7]: "1724169600000-0"
```

Добавим ещё несколько:

```
In [8]: XADD events:users * action logout user_id 42
        XADD events:users * action upload user_id 7 filename report.pdf
        XADD events:users * action click user_id 7 button buy_now
```

7.4.2. XRANGE — прочитать диапазон лога

Это команда для **просмотра истории**. Она доказывает, что Stream — это лог, а не просто очередь.

```
In [9]: XRANGE events:users - +
```

Разбор:

- `-` — минимальный ID (начало потока).

- `+` — максимальный ID (конец потока).

Результат — список всех сообщений с их ID, полями и значениями. Вы увидите, что сообщения **все на месте**, никто их не удалил.

Можно читать диапазон по времени:

```
In [10]: XRANGE events:users 1724169600000 1724169700000
```

7.4.3. `XREAD` — чтение новых сообщений (standalone consumer)

`XREAD` — это команда для чтения одним или несколькими независимыми читателями. Она **не использует группы** и **не требует АСК**.

Синтаксис:

```
In [11]: XREAD [COUNT n] [BLOCK ms] STREAMS имя_потока ID
```

Ключевые параметры:

Параметр	Значение
<code>COUNT 2</code>	Прочитать максимум 2 сообщения за раз
<code>BLOCK 0</code>	Блокировать бесконечно, пока не появятся новые сообщения
<code>BLOCK 5000</code>	Ждать 5 секунд, потом вернуть пустой ответ
<code>STREAMS events:users 0</code>	Читать поток <code>events:users</code> , начиная с ID <code>0</code> (всё с начала)
<code>STREAMS events:users \$</code>	Читать только новые сообщения, появившиеся после вызова команды

Практика:

Откройте **два** терминала с `redis-cli`.

Терминал 1 (Consumer):

```
In [12]: XREAD BLOCK 0 STREAMS events:users $
```

Команда зависнет. Она ждёт новых сообщений (\$).

Терминал 2 (Producer):

```
In [13]: XADD events:users * action purchase user_id 99 amount 1500
```

Терминал 1 мгновенно выведет:

```
In [14]: 1) 1) "events:users"
          2) 1) 1) "1724169800000-0"
            2) 1) "action"
              2) "purchase"
              3) "user_id"
              4) "99"
              5) "amount"
              6) "1500"
```

Важно: После `XREAD` сообщение **не удалено** из потока. Если вы снова выполните `XRANGE events:users - +` — оно там будет.

7.4.4. `XGROUP CREATE` — создать группу потребителей

Группы нужны, когда несколько воркеров делят работу. Группа — это «команда поваров», которые договариваются, кто какой заказ берёт.

```
In [15]: XGROUP CREATE events:users processors $ MKSTREAM
```

Разбор:

- `events:users` — поток.
- `processors` — имя группы.
- `$` — начать отслеживание с новых сообщений (поток уже существует, поэтому `MKSTREAM` не обязателен, но не мешает).
- `MKSTREAM` — создать поток, если его ещё нет.

7.4.5. `XREADGROUP` — чтение внутри группы

Это главная команда для production-воркеров.

```
In [16]: XREADGROUP GROUP processors worker_1 COUNT 1 BLOCK 0 STREAMS events:users >
```

Разбор:

- `GROUP processors worker_1` — читаем от имени группы `processors`, представляясь как `worker_1`.
- `COUNT 1` — взять одно сообщение.
- `BLOCK 0` — ждать бесконечно.
- `STREAMS events:users` — из потока `events:users`.
- `>` — специальный символ: «дай мне сообщения, которые ещё **не были доставлены** никому в этой группе».

Что происходит: Redis запоминает, что сообщение `1724169800000-0` было отдано `worker_1`. Оно переходит в статус **pending** (ожидает подтверждения). Другой воркер, выполнив ту же команду, **не получит** это сообщение — получит следующее.

7.4.6. `XACK` — подтвердить обработку

Когда воркер успешно обработал задачу, он должен сказать Redis: «Я справился, удали из pending».

```
In [17]: XACK events:users processors 1724169800000-0
```

Если воркер упал до `XACK`: Сообщение остаётся в pending-списке группы. Его можно перехватить другому воркеру (через `XPENDING` и `XCLAIM` — продвинутые темы, мы коснёмся их в Модуле 12).

7.4.7. `XPENDING` — посмотреть «зависшие» сообщения

```
In [18]: XPENDING events:users processors
```

Покажет:

- Количество pending-сообщений.
- Минимальный и максимальный ID.
- Список consumer'ов и сколько сообщений висят на каждом.

7.5. Архитектура: почему Stream — это лог, а не очередь

7.5.1. Сравнение на уровне ментальной модели

Очередь задач (Celery + Redis List):

- Вы кладёте задачу.
- Воркер забирает — задача исчезает.
- Цель: выполнить и забыть.
- Аналогия: конвейер на заводе. Деталь прошла станок — её больше нет на конвейере.

Лог событий (Redis Stream / Kafka):

- Вы записываете факт.
- Любое количество сервисов может прочитать этот факт.
- Запись остаётся (пока не истечёт TTL или не превысится лимит).
- Аналогия: газета. Один выпуск. Миллион читателей. Каждый читает самостоятельно. Газета не исчезает после того, как её прочитал первый человек.

7.5.2. Когда что использовать

Сценарий	Технология	Почему
«Отправить email и забыть»	Celery + Redis List	Просто, быстро, не нужна история
«Пользователь загрузил файл — обработать ML-моделью»	Celery + Redis	Задача эфемерна, нужен только результат
«Зафиксировать все клики пользователя для аналитики»	Redis Streams / Kafka	Нужна история, несколько читателей
«Репликация данных между сервисами»	Kafka / Streams	Нужна персистентность и перемотка
«Аудит: кто и когда что сделал»	Streams	Неизменяемый журнал

7.6. Практика: пишем Producer и Consumer на Python

Мы создадим три Python-скрипта:

1. `stream_producer.py` — публикует события через `XADD`.

2. `stream_consumer_simple.py` — читает через `XREAD` (standalone).
3. `stream_consumer_group.py` — читает через `XREADGROUP` с ACK.

Шаг 0. Подготовка

Создайте папку:

```
In [19]: mkdir ~/docker-module7
         cd ~/docker-module7
```

Установите библиотеку:

```
In [20]: pip install redis
```

Или, если вы работаете внутри Docker-контейнера из прошлых модулей, убедитесь, что `redis` есть в `requirements.txt`.

Запустите Redis (если не запущен):

```
In [21]: docker run -d --name redis_stream -p 6379:6379 redis:7-alpine
```

Шаг 1. Producer (`stream_producer.py`)

```
In [22]: import time
         import random
         import redis

         # Подключаемся к Redis
         # Если Redis в Docker на хосте: host="localhost"
         # Если из контейнера внутри docker-compose: host="redis"
         r = redis.Redis(host='localhost', port=6379, decode_responses=True)

         STREAM_NAME = "ml:events"

         def produce_events(count=10):
             """Генерируем события и публикуем их в Stream."""
             actions = ["upload", "preprocess", "train_start", "train_end", "predict"]

             for i in range(count):
                 event = {
```

```

        "event_id": str(i + 1),
        "action": random.choice(actions),
        "user_id": str(random.randint(100, 999)),
        "model": random.choice(["linear", "tree", "neural"]),
        "timestamp": str(time.time())
    }

    # XADD с автогенерацией ID (*)
    # Можно ограничить длину потока: MAXLEN ~ 1000
    msg_id = r.xadd(STREAM_NAME, event)
    print(f"[PRODUCER] Отправлено событие #{i+1}, ID: {msg_id}, action: {event['action']}")
    time.sleep(random.uniform(0.5, 1.5))

    print("[PRODUCER] Все события отправлены.")

if __name__ == "__main__":
    produce_events(10)

```

Запустите:

In [23]: `python stream_producer.py`

Шаг 2. Simple Consumer через XREAD (stream_consumer_simple.py)

```

In [24]: import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)
STREAM_NAME = "ml:events"

def consume_simple():
    """Читаем весь поток с начала и ждём новых."""
    # Начинаем с начала потока (ID "0")
    last_id = "0"

    print("[CONSUMER SIMPLE] Начинаю чтение с начала потока...")

    while True:
        # XREAD с блокировкой на 5 секунд
        # STREAMS ml:events 0 – читать всё с начала (для демо)
        # В production использовали бы "$" для новых сообщений
        response = r.xread({STREAM_NAME: last_id}, block=5000, count=1)

```

```

    if response:
        # response: [['ml:events', [('id', {'field': 'value'}), ...]]]
        for stream_name, messages in response:
            for msg_id, fields in messages:
                print(f"[CONSUMER] ID: {msg_id}")
                print(f"        Данные: {fields}")
                print("-" * 40)
                last_id = msg_id # Сдвигаем указатель
            else:
                print("[CONSUMER] Нет новых сообщений за 5 секунд. Проверяю снов
a...")

if __name__ == "__main__":
    consume_simple()

```

Запустите в отдельном терминале:

In [25]: `python stream_consumer_simple.py`

Наблюдайте: Consumer прочитает все 10 сообщений, которые отправил Producer. Потом будет ждать новых. Если запустите Producer ещё раз — Consumer их тоже прочитает (потому что `last_id` сдвигается).

Шаг 3. Consumer Group (stream_consumer_group.py)

```

In [26]: import time
import sys
import redis

r = redis.Redis(host='localhost', port=6379, decode_responses=True)
STREAM_NAME = "ml:events"
GROUP_NAME = "ml_workers"
CONSUMER_NAME = sys.argv[1] if len(sys.argv) > 1 else "worker_1"

def ensure_group():
    """Создаём группу, если её нет."""
    try:
        r.xgroup_create(STREAM_NAME, GROUP_NAME, id="0", mkstream=True)
        print(f"[{CONSUMER_NAME}] Группа {GROUP_NAME} создана.")
    except redis.exceptions.ResponseError as e:
        if "already exists" in str(e):
            print(f"[{CONSUMER_NAME}] Группа {GROUP_NAME} уже существует.")

```

```

        else:
            raise

def consume_group():
    """Читаем из группы с подтверждением обработки."""
    print(f"[{CONSUMER_NAME}] Выходит на работу...")

    while True:
        # XREADGROUP
        # > — означает: дай мне сообщения, которые ещё не были доставлены в групп
y
        response = r.xreadgroup(
            groupname=GROUP_NAME,
            consumername=CONSUMER_NAME,
            streams={STREAM_NAME: ">"},
            count=1,
            block=5000
        )

        if response:
            for stream_name, messages in response:
                for msg_id, fields in messages:
                    print(f"[{CONSUMER_NAME}] Обработка: {msg_id}")
                    print(f"                Данные: {fields}")

                    # ИМИТАЦИЯ ОБРАБОТКИ
                    time.sleep(2)

                    # ПОДТВЕРЖДАЕМ ОБРАБОТКУ (ACK)
                    r.xack(STREAM_NAME, GROUP_NAME, msg_id)
                    print(f"[{CONSUMER_NAME}] TRUE ACK отправлен: {msg_id}")
                    print("-" * 40)
                else:
                    print(f"[{CONSUMER_NAME}] Очередь пуста. Жду...")

if __name__ == "__main__":
    ensure_group()
    consume_group()

```

Запустите два воркера в двух терминалах:

```

In [27]: # Терминал 1
python stream_consumer_group.py worker_alpha

```

```
# Терминал 2
python stream_consumer_group.py worker_beta
```

Затем запустите Producer ещё раз:

```
In [28]: python stream_producer.py
```

Наблюдайте:

- Сообщения распределяются между `worker_alpha` и `worker_beta` . Одно сообщение не попадает обоим.
- Каждый воркер отправляет `ACK` .
- Если один воркер упадёт (закройте терминал) во время обработки — его сообщение останется в `pending`. Можно проверить:

```
In [29]: docker exec -it redis_stream redis-cli
XPENDING ml:events ml_workers
```

7.7. Подключение из Docker Compose (интеграция с проектом)

Если вы хотите запускать Producer/Consumer внутри контейнеров (как в Модуле 5), `host` в коде должен быть `"redis"` (имя сервиса), а не `"localhost"` .

Пример `docker-compose.yml` для этого модуля:

```
In [30]: services:
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

  producer:
    build: .
    depends_on:
      - redis
    command: python stream_producer.py

  consumer:
    build: .
    depends_on:
      - redis
```

```
command: python stream_consumer_group.py docker_consumer
deploy:
  replicas: 2 # Два воркера!
```

7.8. Итоги модуля: чек-лист

- ☐ Понимаю, почему классические списки Redis (`LPUSH` / `BRPOP`) не подходят для надёжных систем.
- ☐ Знаю, что такое **Append-Only Log**: журнал, куда пишут только в конец, а старые записи не изменяют.
- ☐ Понимаю структуру ID в Redis Streams: `timestamp-sequencenumber` .
- ☐ Умею добавлять сообщения через `XADD` (включая автогенерацию ID через `*`).
- ☐ Умею читать историю через `XRANGE` .
- ☐ Умею читать новые сообщения через `XREAD` (с `BLOCK` и `$`).
- ☐ Понимаю разницу между `XREAD` (standalone, без ACK) и `XREADGROUP` (внутри группы с дедупликацией).
- ☐ Умею создавать группу через `XGROUP CREATE` .
- ☐ Знаю, что символ `>` в `XREADGROUP` означает «только новые для группы».
- ☐ Умею подтверждать обработку через `XACK` .
- ☐ Умею проверять «зависшие» сообщения через `XPENDING` .
- ☐ Понимаю ключевое архитектурное отличие: в Stream сообщение **не удаляется** после прочтения, в отличие от классической очереди.
- ☐ Написал и запустил Producer и Consumer на Python (`redis-py`).
- ☐ Увидел распределение нагрузки между несколькими Consumer'ами в группе.

В следующем модуле мы познакомимся с **Celery** — высокоуровневой абстракцией над брокерами, которая скрывает команды `XADD` / `XREAD` за декораторами Python-функций. Мы научимся ставить задачи в очередь из API и выполнять их фоновыми воркерами.